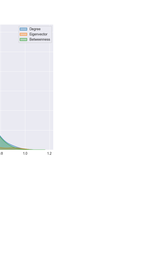
A feature only model trained with node features as independent, and mean prediction over using that node's own attributes. It contains 4 GNN treatments models as dependent and they are embedded in a network where those nodes share similar characteristics. Through message passing, each node receives information from its neighbors, and after several layers, each node's representation captures not just its own features but also the structural context of its local neighborhood. The most common problem of GNNs is homophily-driven classification. In sociological terms, similar nodes connect more easily than dissimilar ones, meaning standard feature-based classifiers can achieve high accuracy by simply learning to predict the majority class within each cluster. However, a GNN trained on a diplomatic network might predict that a country will adopt treatment because its neighbors adopted it, even if the country's own attributes show similar economic conditions that independently drive policy adoption. In this case, the GNN's performance is boosted by causing connection for connection causing behavior.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, mean_squared_error

import torch
import torch_geometric as tg
import torch.nn.functional as F

from datetime import date

# Reproducibility
np.random.seed(123)
torch.manual_seed(123)

# 01. Generate a Synthetic Network (Structural Block Model)

# Step 1: Parameters for community sizes and connection probabilities

# Three communities (blocks)
block_sizes = [500, 250, 250]
num_nodes = sum(block_sizes)
num_edges = int(0.0_5 * num_nodes)

# SBM probability matrix
# Higher values on the diagonal mean stronger within-community ties
# Lower values off-diagonal mean weaker between-community ties
P = np.array([
    [0.0_4, 0.0_1, 0.0_1],
    [0.0_1, 0.0_4, 0.0_1],
    [0.0_1, 0.0_1, 0.0_4]
])

# Step 2: Generate the graph with SBM
# stochastic_block_model takes sizes and probs, creates edges probabilistically
G = nx.stochastic_block_model(block_sizes, P, seed=123)
print(f"Initial edges: {G.number_of_edges()}")
edge_list = np.data(f"arr0:i:2;", edges=G, ""+sep="\n")
with open("edges.txt", "wb") as f:
    print(edge_list.tobytes())

# reconstruct the graph
G = nx.from_pandas_edgelist(edge_list, source="u", target="v")
print("Reconstructed graph summary:")
print(f"Nodes: {G.number_of_nodes()}")
print(f"Edges: {G.number_of_edges()}")

# Compute three centrality
deg_cent = nx.degree_centrality(G)
deg_series = np.array([deg_cent[i]for i in range(len(G))])
deg_summary = pd.Series(deg_series).describe()
print("Degree centrality:")
print(deg_summary)

# Betweenness centrality
betw = nx.betweenness_centrality(G, k=200, seed=123)
betw_series = np.array([betw.get(i,0)for i in range(len(G))])
betw_summary = pd.Series(betw_series).describe()
print("Betweenness centrality:")
eig_cent = np.array([eig_cent[i]for i in range(len(G))])
eig_series = np.array([eig_cent[i]for i in range(len(G))])
eig_summary = pd.Series(eig_series).describe()
print("Eigenvector centrality:")
print(eig_summary)

# Visualize degree distribution
plt.figure(figure=(8,5))
sns.kdeplot(deg_series, label="Degree", fill=True, alpha=0.3)
sns.kdeplot(betw_series, label="Betweenness", fill=True, alpha=0.3)
sns.kdeplot(eig_series, label="Betweenness", fill=True, alpha=0.3)

plt.title("Distribution of Centrality Measures")
plt.xlabel("Centrality")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()

# print top 10 nodes
# transform degree to dictionary
deg_dict = {i: deg_series[i]}
top_deg = sorted(deg_dict.items(), key=lambda x: x[1], reverse=True)[:10]
betw_dict = {i: betw_series[i]}
top_betw = sorted(betw_dict.items(), key=lambda x: x[1], reverse=True)[:10]

print("Degree:")
for node, value in top_deg:
    print(f"  {node}: {value:.6f}")
print("Betweenness:")
for node, value in top_betw:
    print(f"  {node}: {value:.6f}")

# save centrality to centralities.csvnodes=
centrality = pd.DataFrame({'node': range(len(G)), 'degree': deg_series, 'betweenness': betw_series, 'eigenvector': eig_series})
centrality.to_csv('centralities.csv', index=False)
print("Saved centralities to centralities.csv")

# Convert to PyTorch Geometric format
edge_index = torch.tensor(edge_list[['u', 'v'].values.T, dtype=torch.long)
x = torch.ones(num_nodes, 1)
data = tg.Data(x=x, edge_index=edge_index)
print("Graph data:")
print(f"  Nodes: {data.num_nodes}")
print(f"  Edges: {data.num_edges}")
print(f"  Node features: {data.x.shape}")
print(f"  Edge index shape: {data.edge_index.shape}")

# Save for later
torch.save(data, 'graph.pt')
print("Saved graph to graph.pt")
```

```
Initial edges: 1250
Reconstructed graph summary:
Nodes: 1000
Edges: 1249

Degree centrality:
count    1000.000000
mean        0.002002
std         0.002019
min         0.000000
50%         0.001000
75%         0.003000
max         0.009000
dtype: float64

Eigenvector centrality:
count    1000.000000
mean        0.000000
std         0.000000
min         0.000000
50%         0.000000
75%         0.000000
max         0.000000
dtype: float64
```




```
Degree:
  0: 0.009000
  1: 0.008000
  2: 0.008000
  3: 0.008000
  4: 0.008000
  5: 0.008000
  6: 0.008000
  7: 0.008000
  8: 0.008000
  9: 0.008000
Betweenness:
  0: 0.000000
  1: 0.000000
  2: 0.000000
  3: 0.000000
  4: 0.000000
  5: 0.000000
  6: 0.000000
  7: 0.000000
  8: 0.000000
  9: 0.000000
Saved centralities to centralities.csv
Graph data:
  Nodes: 1000
  Edges: 1250
  Node features: (1000, 1)
  Edge index shape: torch.Size([2, 1250])
Saved graph to graph.pt
```

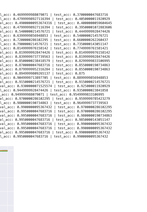
```
   centrality  Node     Value
0      degree   304  0.009000
1      degree   305  0.008000
2      degree   306  0.008000
3      degree   307  0.008000
4      degree   308  0.008000
5      degree   309  0.008000
6      degree   310  0.008000
7      degree   311  0.008000
8      degree   312  0.008000
9      degree   313  0.008000
10 Eigenvector   304  0.073735
11 Eigenvector   305  0.067000
12 Eigenvector   306  0.064040
13 Eigenvector   307  0.062000
14 Eigenvector   308  0.061537
15 Eigenvector   309  0.060000
16 Eigenvector   310  0.058000
17 Eigenvector   311  0.052750
18 Eigenvector   312  0.052000
19 Eigenvector   313  0.051511
20 Betweenness   530  0.000019
21 Betweenness   531  0.000000
22 Betweenness   532  0.000000
23 Betweenness   533  0.000000
24 Betweenness   534  0.000139
25 Betweenness   535  0.000000
26 Betweenness   536  0.000000
27 Betweenness   537  0.000000
28 Betweenness   538  0.000000
29 Betweenness   539  0.000000
```

```
# 02 community detection + evaluation
lovain_comm = m.louvain(seed=123)
lovain_communities=lg2.count_communities(lg2, seed=123)

# Convert communities into a node -> community_id vector
lovain_labels = np.zeros(num_nodes, dtype=int)
for comm_id in lovain_communities:
    for node in lovain_communities[comm_id]:
        lovain_labels[node] = comm_id
    print(f"  Community {comm_id}: {len(lovain_communities[comm_id])} nodes")

print("Community detection summary:")
print(f"  Number of louvain communities: {len(lovain_communities)}")

lovain_sizes = [len(lovain_communities[i])for i in lovain_communities]
print(f"  Louvain community sizes (first 10): {lovain_sizes[:10]}")

# Compare to ground-truth block structure
# 0-499 = Block 0, 500-749 = Block 1, 750-999 = Block 2
# Step 2: Create ground-truth labels (community membership)
true_labels = np.concatenate([
    np.zeros(block_sizes[0], dtype=int),
    np.ones(block_sizes[1], dtype=int),
    2 * np.ones(block_sizes[2], dtype=int)
])

# Create synthetic node features correlated with communities
num_features = 16

centers = np.array([
    [1, 0, 0, 0, 0, 0, 0, 0],  # community 0 center
    [0, 1, 0, 0, 0, 0, 0, 0],  # community 1 center
    [0, 0, 1, 0, 0, 0, 0, 0]   # community 2 center
])  # shape: (3, 8)

noise_std = 0.8

X = np.zeros((num_nodes, num_features), dtype=float)
X[true_labels == 0] = centers[0] + np.random.normal(0, noise_std, size=(np.sum(true_labels == 0), num_features))
X[true_labels == 1] = centers[1] + np.random.normal(0, noise_std, size=(np.sum(true_labels == 1), num_features))
X[true_labels == 2] = centers[2] + np.random.normal(0, noise_std, size=(np.sum(true_labels == 2), num_features))

# Convert with torch
data.louvain_labels = torch.tensor(lovain_labels, dtype=torch.long)
data.true_labels = torch.tensor(true_labels, dtype=torch.long)
data.x = torch.tensor(X, dtype=torch.float)

# Split (stratified by community)
from sklearn.model_selection import train_test_split
idx = np.arange(num_nodes)
idx_train, idx_test = train_test_split(idx, test_size=0.2, stratify=true_labels, random_state=123)
idx_train, idx_val = train_test_split(idx_train, test_size=0.1, stratify=true_labels[idx_train], random_state=123)

data.train_mask = torch.zeros(num_nodes, dtype=torch.bool)
data.val_mask = torch.zeros(num_nodes, dtype=torch.bool)
data.test_mask = torch.zeros(num_nodes, dtype=torch.bool)
data.train_mask[torch.tensor(idx_train)] = True
data.val_mask[torch.tensor(idx_val)] = True
data.test_mask[torch.tensor(idx_test)] = True

print("Feature shape:", data.x.shape)
print("Train/val/test:", data.train_mask.sum(), data.val_mask.sum(), data.test_mask.sum)

# Save
torch.save(data, 'graph_louvain.pt')
print("Saved to graph_louvain.pt")
```

```
  Community 0: 500 nodes
  Community 1: 250 nodes
  Community 2: 250 nodes
Community detection summary:
  Number of louvain communities: 3
  Louvain community sizes (first 10): [500, 250, 250]
```

```
# 03 GNN
# Split data into train, val, and test
# We do a random permutation of nodes
perm = torch.randperm(num_nodes)

train_size = int(0.6 * num_nodes)
val_size = int(0.2 * num_nodes)
test_size = num_nodes - train_size - val_size

train_idx = perm[:train_size]
val_idx = perm[train_size:train_size + val_size]
test_idx = perm[train_size + val_size:]

# Create boolean masks
data.train_mask = torch.zeros(num_nodes, dtype=torch.bool)
data.val_mask = torch.zeros(num_nodes, dtype=torch.bool)
data.test_mask = torch.zeros(num_nodes, dtype=torch.bool)

data.train_mask[train_idx] = True
data.val_mask[val_idx] = True
data.test_mask[test_idx] = True

print("Train/val/test nodes:", data.train_mask.sum(), data.val_mask.sum(), data.test_mask.sum()
# For logistic regression on features only
X = data.x.numpy()
y = data.louvain_labels.numpy()

X_train = X[train_idx]
y_train = y[train_idx]
X_test = X[test_idx]
y_test = y[test_idx]

# Train logistic regression
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=1000, random_state=123)
lr.fit(X_train, y_train)

# Predictions
y_pred_train = lr.predict(X_train)
y_pred_test = lr.predict(X_test)

# Accuracy
from sklearn.metrics import accuracy_score, f1_score
train_acc = accuracy_score(y_train, y_pred_train)
test_acc = accuracy_score(y_test, y_pred_test)

print("\nBaseline (features-only logistic regression):")
print("  Train accuracy:", train_acc,)
print("  Test accuracy:", test_acc,)

# Convert the graph to a sparse adjacency matrix
A = nx.to_scipy_sparse_array(
    G,
    nodelist=range(num_nodes),
    dtype=float,
    weight=None,
    format='csr'
)

print("\nAdjacency matrix:")
print("  Shape:", A.shape)
print("  Nonzeros:", A.nnz)

# Add self-loops and compute normalized adjacency (sparse)
A = A + sp.eye(num_nodes, format="csr", dtype=A.dtype)
A_hat = A

deg_inv_sqrt = np.array(A_hat.sum(axis=1).flatten)
deg_inv_sqrt = 1.0 / np.sqrt(deg_inv_sqrt)

D_inv_sqrt = sp.diags(deg_inv_sqrt.astype(np.float32), format="csr")
A_norm = D_inv_sqrt @ A_hat @ D_inv_sqrt

# Step 3: Convert data to PyTorch tensors
A_norm_coo = A_norm.tocoo()
ij = torch.tensor(np.vstack([A_norm_coo.row, A_norm_coo.col]),

# Convert A_norm to a torch sparse tensor
A_norm_torch = torch.sparse_coo_tensor(
    ij,
    torch.tensor(A_norm_coo.data, dtype=torch.float32),
    A_norm.shape
)

# Create X_norm (identity for now)
X_norm = torch.eye(num_nodes, dtype=torch.float32)

# Convert labels to torch tensor
y = torch.tensor(y, dtype=torch.long)

# Simple 2-layer GCN
class GCN(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, out_dim)

    def forward(self, A, X):
        ax = A @ X
        h = self.fc1(ax)
        h = torch.relu(h)
        ah = A @ h
        out = self.fc2(ah)
        return out

model = GCN(in_dim=16, hidden_dim=32, out_dim=3)

optimizer = torch.optim.Adam(list(model.parameters()) + list(lr.parameters()), lr=0.01, weight_decay=5e-4)

# Step 4: Training loop (complexity grows with depth)
# Track losses for plotting
epochs = 50

# Track training history so we can visualize performance across epochs
epoch_list = []
train_loss_list = []
train_acc_history = []
test_acc_history = []

for epoch in range(1, epochs + 1):

    # Forward pass
    optimizer.zero_grad()
    H1 = torch.sparse.mm(A_norm, W1)
    H1 = torch.relu(H1)

    ZW = torch.mm(H1, W2)
    logits = torch.sparse.mm(A_norm, ZW)

    # Loss on training nodes only
    loss = F.cross_entropy(logits[train_idx], y_train_idx,)

    # Backprop + update
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Accuracy (transductive)
    preds = torch.argmax(logits, dim=1)

    train_acc = (preds[train_idx] == y_train_idx.float().mean().item()
    val_acc = (preds[val_idx] == y_val_idx.float().mean().item()
    test_acc = (preds[test_idx] == y_test_idx.float().mean().item()

    print("Epoch", epoch, "| Loss:", float(loss.detach().item()),
          "| Train acc:", train_acc, "| Val acc:", val_acc, "| Test acc:", test_acc)

    epoch_list.append(epoch)
    train_loss_list.append(float(loss.detach().item()))
    train_acc_history.append(train_acc)
    val_acc_history.append(val_acc)
    test_acc_history.append(test_acc)

# Supervised training loop
# This shows how train/val/test splits work in practice
# We also print loss occasionally so students can compare optimization vs accuracy.

history_df = pd.DataFrame({
    'epoch': epoch_list,
    'train_loss': train_loss_list,
    'train_acc': train_acc_history,
    'val_acc': val_acc_history,
    'test_acc': test_acc_history
})

# Accuracy complexity plot
plt.figure(
plt.plot(history_df["epoch"], history_df["train_acc"], label="Train accuracy")
plt.plot(history_df["epoch"], history_df["val_acc"], label="Validation accuracy")
plt.plot(history_df["epoch"], history_df["test_acc"], label="Test accuracy")
plt.title("GCN accuracy across epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()

# Loss plot
plt.figure(
plt.plot(history_df["epoch"], history_df["loss"], label="Training loss")
plt.title("GCN Training loss across epochs")
plt.xlabel("Epoch")
plt.ylabel("Cross-entropy loss")
plt.legend()
plt.tight_layout()
plt.show()

# Confusion Matrix + Comparison
# Compare baseline (features-only) vs GCN (features + graph)
# View a confusion matrix-like table for the GCN test predictions

pred_np = preds.detach().cpu().numpy()

test_truth = y[test_idx]
test_pred_gcn = pred_np[test_idx]

test_acc_gcn = accuracy_score(test_truth, test_pred_gcn)

print("\n" "Model comparison")
print("  Baseline LR test accuracy:", test_acc_lr)
print("  GCN test accuracy:", test_acc_gcn)

confusion_tbl = pd.crosstab(
    pd.Series(test_truth, name="True"),
    pd.Series(test_pred_gcn, name="Predicted")
)

print("\nGCN confusion table (test set):")
print(confusion_tbl)
print(confusion_tbl)

# Compare to baseline logistic regression
print("\nBaseline (logistic regression)")
Test accuracy: 0.675
```

```
Adjacency matrix:
  Shape: (1000, 1000)
  Nonzeros: 7500

Epoch 1 | Loss: 1.0986180019122176 | Train acc: 0.4200847699193394 | val_acc: 0.4999999880791 | Test acc: 0.7300000190734863
Epoch 2 | Loss: 1.0986180019122176 | Train acc: 0.4200847699193394 | val_acc: 0.4999999880791 | Test acc: 0.7300000190734863
Epoch 3 | Loss: 0.9780140137158843 | Train acc: 0.4433333277893353 | val_acc: 0.4999999880791 | Test acc: 0.6800000190734863
Epoch 4 | Loss: 0.8574080276489258 | Train acc: 0.5666666674232483 | val_acc: 0.5999999880791 | Test acc: 0.5799999952316284
Epoch 5 | Loss: 0.8024620056152344 | Train acc: 0.6666666674232483 | val_acc: 0.6999999880791 | Test acc: 0.6499999761581421
Epoch 6 | Loss: 0.8024620056152344 | Train acc: 0.6666666674232483 | val_acc: 0.5999999880791 | Test acc: 0.5499999761581421
Epoch 7 | Loss: 0.7469099760055542 | Train acc: 0.7666666674232483 | val_acc: 0.6999999880791 | Test acc: 0.6499999761581421
Epoch 8 | Loss: 0.6913559246063232 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 9 | Loss: 0.7007537484264183 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 10 | Loss: 0.7101516127204895 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 11 | Loss: 0.7195494771003723 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 12 | Loss: 0.7289473414802551 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 13 | Loss: 0.7383452057600403 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 14 | Loss: 0.7477430701398277 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 15 | Loss: 0.7571409344196129 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 16 | Loss: 0.7665387988090515 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 17 | Loss: 0.7759366631984329 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 18 | Loss: 0.7853345275878906 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 19 | Loss: 0.7947323918771362 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 20 | Loss: 0.8041302561759949 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 21 | Loss: 0.8135281205749512 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 22 | Loss: 0.8229259848690033 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 23 | Loss: 0.8323238492012024 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 24 | Loss: 0.8417217135429382 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 25 | Loss: 0.8511195778846359 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 26 | Loss: 0.8605174422264099 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 27 | Loss: 0.8699153065681458 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 28 | Loss: 0.8793131709098816 | Train acc: 0.8666666674232483 | val_acc: 0.7999999880791 | Test acc: 0.7399999761581421
Epoch 29 | Loss: 0.8887110352516174 | Train acc: 0.7666666674232483 | val_acc: 0.6999999880791 | Test acc: 0.6499999761581421
Epoch 30 | Loss: 0.8981088995933533 | Train acc: 0.7666666674232483 | val_acc: 0.6999999880791 | Test acc: 0.6499999761581421
```



```
Model comparison
  Baseline LR test accuracy: 0.675
  GCN test accuracy: 0.79
GCN confusion table (test set):
Predicted  0  1  2
True
0         87  0  0
1          0 50  0
2          0 13 50
Predicted  0  1  2
True
0         87  0  0
1          0 50  0
2          0 13 50
Baseline (logistic regression)
  Test accuracy: 0.675
```

The GCN outperforms the baseline logistic regression (0.80 vs 0.675), suggesting that incorporating network structure through message passing provides additional predictive signal beyond node features alone. The confusion matrix shows perfect classification of class 0 (87/87 correct) and class 1 (50/50 correct), with all errors in class 2 (13 nodes predicted as class 1). The significant node features and graph with segregated across communities. However, in real-world networks where node features are noisy and community boundaries are less clear, the GCN's ability to leverage both features and structure would likely produce a more substantial performance gap over a feature-only baseline.